

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSIGNMENT REPORT

Object-Oriented Analysis and UML-Based Design of an
Autonomous Drone Disaster Response and Rescue System (ADDRRS)

Field	Details
Course Name	Object Oriented Analysis and Design
Course Code	CSA11
Assignment Weightage	100%
Total Marks	100
Student Name	M Nithishkumaran
Register Number	192320037
Date of Submission	01.09.2026

Code of Conduct Declaration

I **M Nithishkumaran (192320037)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **M Nithishkumaran**

1. Problem Analysis and Requirement Summary

1.1 Problem Statement

The Autonomous Drone Disaster Response and Rescue System (ADDRRS) is proposed to assist emergency response teams during natural and man-made disasters such as floods, earthquakes, building collapses, and wildfires. The system enables authorized emergency personnel to register disaster incidents, delineate affected zones, assign autonomous drones for reconnaissance, and coordinate rescue missions. Drones equipped with high-resolution cameras and environmental sensors autonomously survey disaster zones, capture visual and sensor data, and transmit this information in real time to a central command system, which analyses the data to detect victims, assess hazards, and trigger alerts for emergency teams.

The primary objective of the system is to reduce response time, enhance situational awareness in hazardous or inaccessible areas, and coordinate the safe and efficient deployment of autonomous and human resources during disaster response operations.

1.2 Requirement Summary

From the given scenario, the functional and non-functional requirements are summarised as follows.

Functional Requirements

- Register disaster incidents and define geographic boundaries of affected zones.
- Assign, dispatch, and monitor autonomous drones for reconnaissance missions.
- Plan optimal flight routes considering zone geometry and hazards.
- Capture and transmit images, video, and environmental sensor readings in real time.
- Automatically detect potential victims and generate alerts/notifications.
- Coordinate emergency team dispatch to confirmed victim locations.
- Track mission progress and generate mission completion reports.
- Manage drone fleet health (battery, faults) and support autonomous return-to-base.
- Maintain secure authentication and role-based access for all users.

Non-Functional Requirements

- Real-time or near-real-time communication between drones and the command center.
- High reliability and fault tolerance for autonomous drone operation.
- Scalability to support multiple simultaneous disaster zones and drone fleets.
- Security of transmitted data and role-based authorization.
- Maintainability and extensibility of the software architecture.

1.3 Object Identification (Preliminary)

An initial noun-phrase analysis of the problem statement was performed to identify candidate objects, which were then filtered against relevance, redundancy, and vagueness criteria. The surviving candidate objects — Drone, Sensor, DisasterIncident, DisasterZone, RescueMission,

Victim, EmergencyTeam, Alert, SensorData, Route, MissionReport, and User (and its specializations) — form the basis of the class model developed in Section 5.

2. Actor Identification

Actors represent roles that interact with the ADDRRS system boundary. The scenario identifies both human actors and autonomous/external system actors.

Actor	Type	Description
Emergency Coordinator	Primary (Human)	Registers incidents, defines disaster zones, assigns drones, manages missions and resources.
Drone Operator	Primary (Human)	Monitors live drone status, plans/approves routes, intervenes during faults.
Emergency Team Member	Primary (Human)	Receives alerts, is dispatched to victim locations, performs physical rescue and updates status.
Autonomous Drone	Primary (System/Autonomous)	Executes missions autonomously: navigates, captures sensor data, transmits telemetry, self-manages faults.
System Administrator	Secondary (Human)	Manages user accounts, roles, and system configuration.
GPS / Navigation System	Secondary (External System)	Supplies positioning data used for route planning and navigation.
Weather Information System	Secondary (External System)	Supplies environmental/weather data used in zone risk assessment.

3. Use Case Descriptions

The following sections describe the core use cases that most directly define system behaviour. Supporting use cases are summarised in the Use Case Diagram (Section 4).

UC-01: Register Disaster Incident

Use Case ID	UC-01
Use Case Name	Register Disaster Incident
Actors	Emergency Coordinator
Preconditions	Coordinator is authenticated and logged in.
Main Flow	1. Coordinator opens 'New Incident' form. 2. Coordinator enters incident type, severity, timestamp and initial location. 3. System validates and creates a DisasterIncident record. 4. System prompts coordinator to define the affected zone boundary.

Alternate / Exception Flow	If mandatory data is missing, the system displays a validation error and the incident is not created.
Postconditions	A new DisasterIncident exists in the system and is available for zone definition and drone assignment.

UC-02: Assign Drone to Mission

Use Case ID	UC-02
Use Case Name	Assign Drone to Mission
Actors	Emergency Coordinator, Autonomous Drone
Preconditions	A DisasterZone has been defined; at least one drone is available (Idle state).
Main Flow	1. Coordinator selects an available drone. 2. Coordinator creates a RescueMission linked to the zone. 3. System (via <<include>> Plan Route) computes an optimal flight path. 4. Coordinator confirms and dispatches the drone.
Alternate / Exception Flow	If no drone is available, the system queues the mission and notifies the coordinator when a drone is freed.
Postconditions	The selected drone transitions to the Assigned state and a RescueMission is created.

UC-03: Collect and Transmit Sensor Data / Detect Victim

Use Case ID	UC-03
Use Case Name	Collect Sensor Data, Assess Zone & Detect Victim
Actors	Autonomous Drone
Preconditions	Drone is In-Flight over an assigned disaster zone.
Main Flow	1. Drone captures images/video (Camera) and environmental readings (EnvironmentalSensor). 2. Drone transmits SensorData to the Command Center. 3. Command Center analyses data (<<include>> Assess Disaster Zone). 4. If a victim signature is detected, the system executes Generate Alert (<<include>>).
Alternate / Exception Flow	If transmission fails, data is buffered onboard and retransmitted once connectivity is restored (fault-tolerance requirement).
Postconditions	SensorData is persisted; if a victim was detected, an Alert has been generated and emergency team notified.

UC-04: Manage Rescue Mission and Generate Report

Use Case ID	UC-04
Use Case Name	Manage Rescue Mission
Actors	Emergency Coordinator, Emergency Team Member
Preconditions	A RescueMission is active and an alert has been raised or zone assessment is complete.
Main Flow	1. Coordinator/Team monitors mission progress. 2. Emergency Team is dispatched and performs rescue. 3. Team updates victim/mission status. 4. On completion, the system executes Generate Mission Completion Report (<<include>>).
Alternate / Exception Flow	If the mission is aborted (e.g., worsening conditions), the coordinator records the reason and the mission is marked Cancelled.
Postconditions	Mission status is updated to Completed (or Cancelled) and a MissionReport is generated.

UC-05: Handle Drone Failure / Return-to-Base

Use Case ID	UC-05
Use Case Name	Handle Drone Failure
Actors	Autonomous Drone, Drone Operator
Preconditions	Drone is In-Flight and executing a mission.
Main Flow	1. Drone's onboard controller detects a fault or critically low battery. 2. Drone autonomously aborts scanning and initiates Return-to-Base. 3. Drone Operator is notified and may intervene manually. 4. Drone lands and enters Charging/Maintenance state.
Alternate / Exception Flow	If the fault prevents safe return, the drone performs an emergency landing at the nearest safe coordinate and alerts the operator.
Postconditions	Drone is safely grounded; mission is reassigned to another available drone if incomplete.

4. Use Case Diagram

The diagram represents the primary and secondary actors, system use cases, and their include/extend relationships within the ADDRRS system boundary.

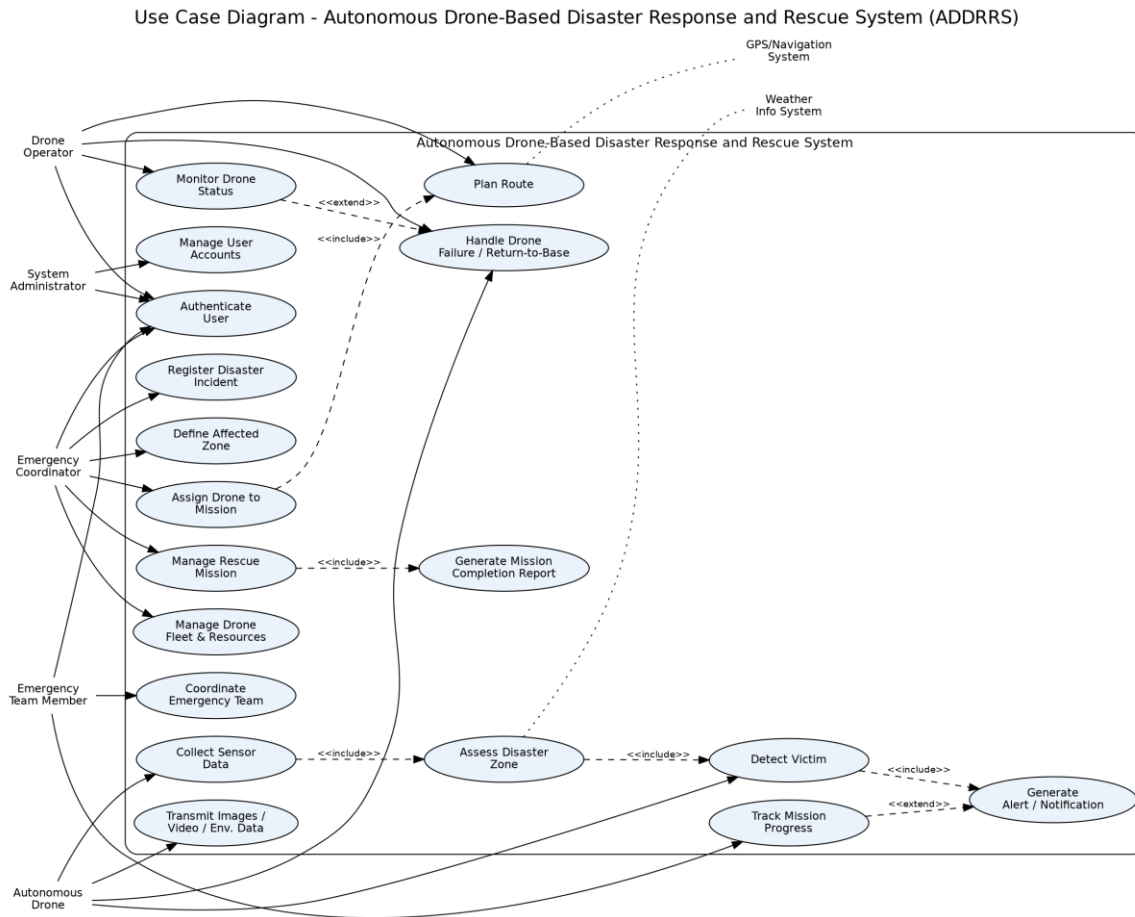


Figure 1: Use Case Diagram — ADDRRS

5. Object and Class Identification

Classes were derived from the noun-phrase/candidate-object analysis in Section 1.3, refined by grouping related attributes and behaviours and eliminating duplicate or irrelevant candidates. Each class was evaluated for a single, well-defined responsibility (high cohesion).

Class	Category	Key Attributes	Key Responsibilities (Methods)
User (abstract)	Actor/Boundary	userID, name, role, credentials	login(), logout()
EmergencyCoordinator	Actor	coordinatorID	registerIncident(), defineZone(), assignDrone()
DroneOperator	Actor	licenseID	monitorDrone(), planRoute()
EmergencyTeamMember	Actor	specialization	receiveAlert(), updateStatus()

Class	Category	Key Attributes	Key Responsibilities (Methods)
Drone	Entity	droneID, status, batteryLevel, currentLocation	takeOff(), land(), navigateTo(), transmitData(), returnToBase()
Sensor (abstract)	Entity	sensorID, sensorType	capture()
Camera / EnvironmentalSensor	Entity	resolution / unit	captureImage(), captureVideo() / readTemperature(), readAirQuality()
DisasterIncident	Entity	incidentID, type, severity, timestamp	registerIncident(), updateSeverity()
DisasterZone	Entity	zoneID, boundaryCoordinates, riskLevel	defineBoundary(), assessRisk()
RescueMission	Control/Entity	missionID, status, priority	assignDrone(), assignTeam(), updateStatus(), completeMission()
Route	Entity	routeID, waypoints, estimatedTime	calculateOptimalPath()
Victim	Entity	victimID, location, status	updateStatus()
EmergencyTeam	Entity	teamID, specialization	addMember(), dispatch()
Alert	Control/Entity	alertID, severity, message	generate(), notify()
SensorData	Entity	dataID, timestamp, value	store()
MissionReport	Entity	reportID, summary, outcome	generateSummary()
CommandCenter	Control (Singleton)	instance	getInstance(), coordinateMission()

6. Class Diagram

The class diagram presents the static structure of ADDRRS, including inheritance through the User and Sensor hierarchies, composition between Drone and its Sensors, aggregation between EmergencyTeam and its members, and directed associations with multiplicities.

Class Diagram - ADDR5

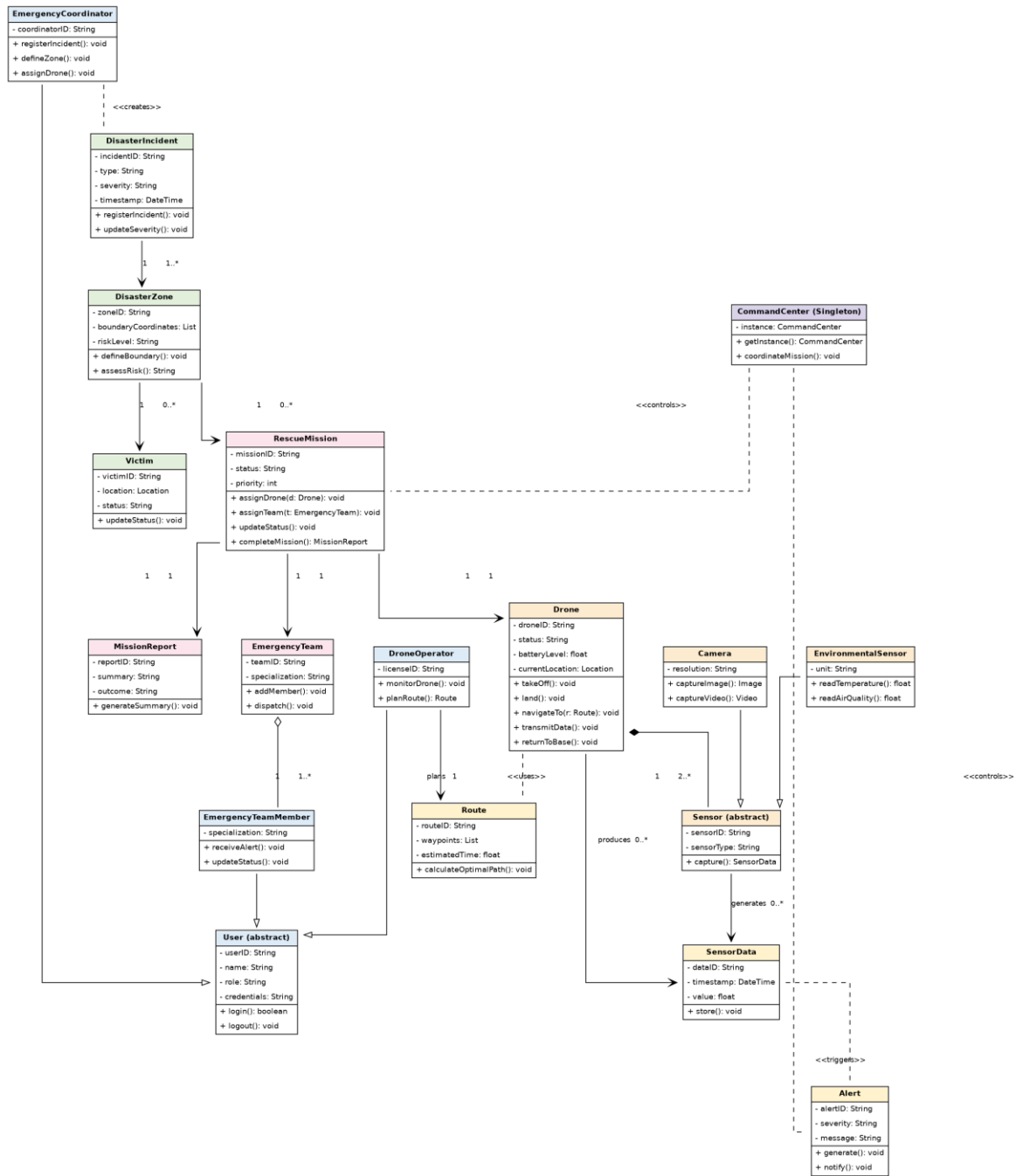


Figure 2: Class Diagram — ADDR5

7. Sequence / Interaction Diagram

The sequence diagram below traces the interaction 'Detect Victim & Coordinate Rescue Mission' — from incident registration, through autonomous drone data capture and transmission, victim detection, alert generation, and emergency-team dispatch, to mission completion and reporting.

This scenario was selected because it exercises the majority of core classes and confirms consistency with the corresponding use cases (UC-02, UC-03, UC-04).

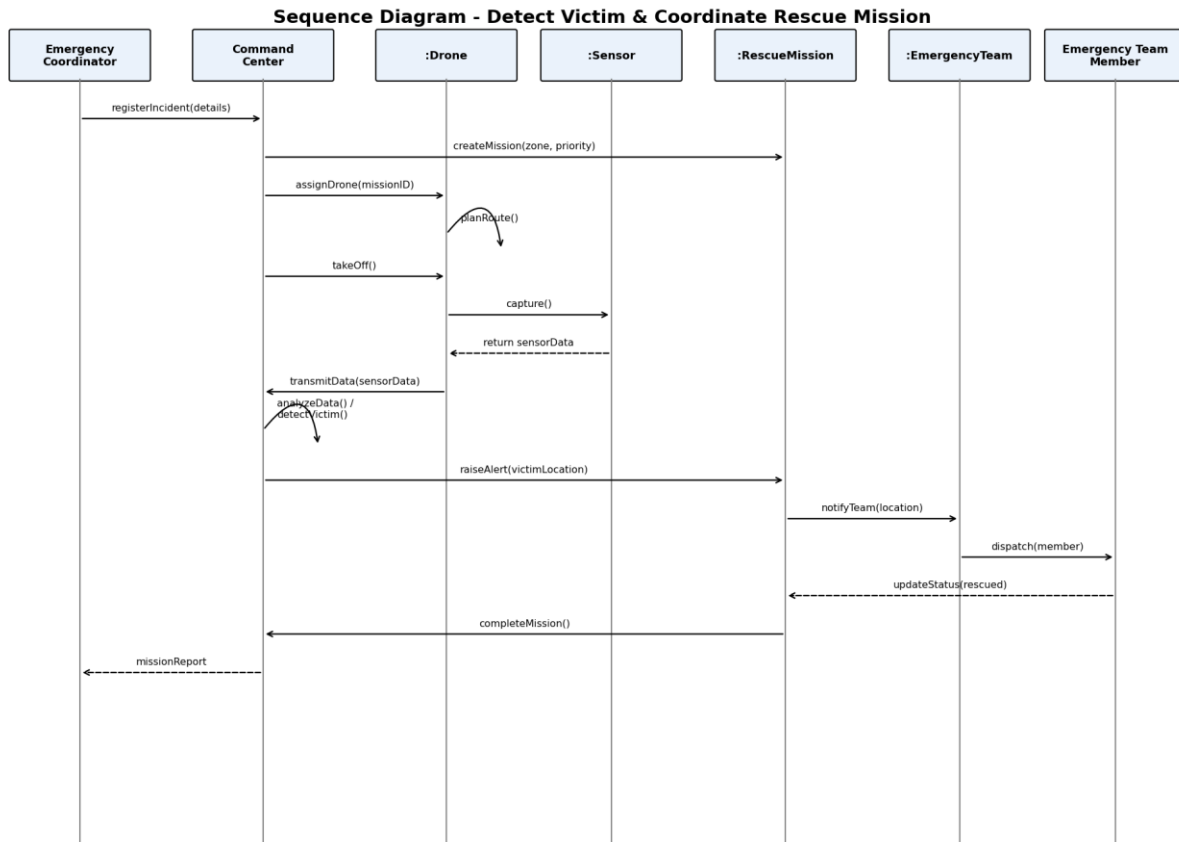


Figure 3: Sequence Diagram — Detect Victim & Coordinate Rescue Mission

8. Activity Diagram

The activity diagram models the end-to-end workflow of a disaster-response mission, including the decision points for victim detection and drone fault handling, and the iterative zone-scanning loop that continues until the assigned zone is fully covered.

Activity Diagram - Disaster Response Mission Workflow

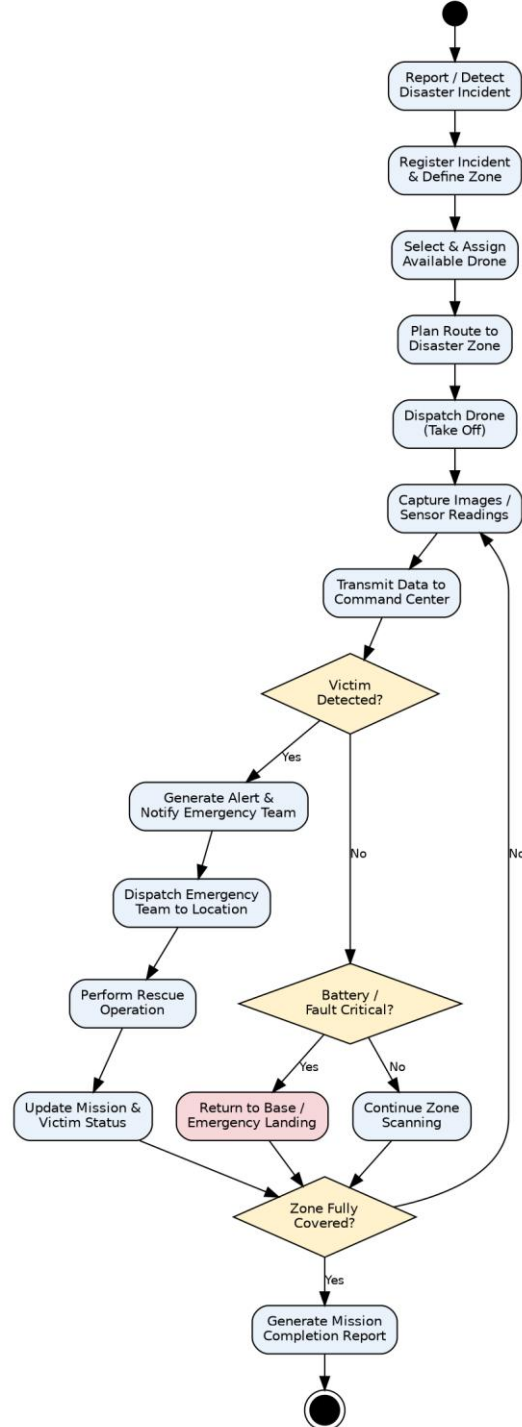


Figure 4: Activity Diagram — Disaster Response Mission Workflow

9. State Transition Diagram

The Drone class exhibits the richest lifecycle in the system and was chosen as the subject of the state transition diagram. States range from Idle through Assigned, Route Planning, In-Flight/Scanning, Transmitting Data, Hovering (on victim detection), Fault handling, Returning to Base, and Charging/Maintenance.

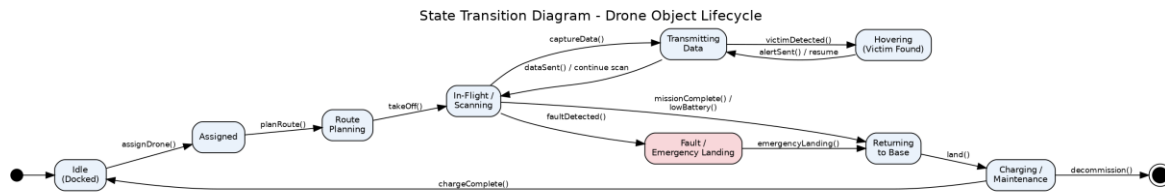


Figure 5: State Transition Diagram — Drone Object Lifecycle

10. Package Diagram

The system is structured into four layers — Presentation, Application/Control, Domain/Entity, and Data/Persistence — to promote low coupling between layers and high cohesion within each package.

Package Diagram - ADDRRS

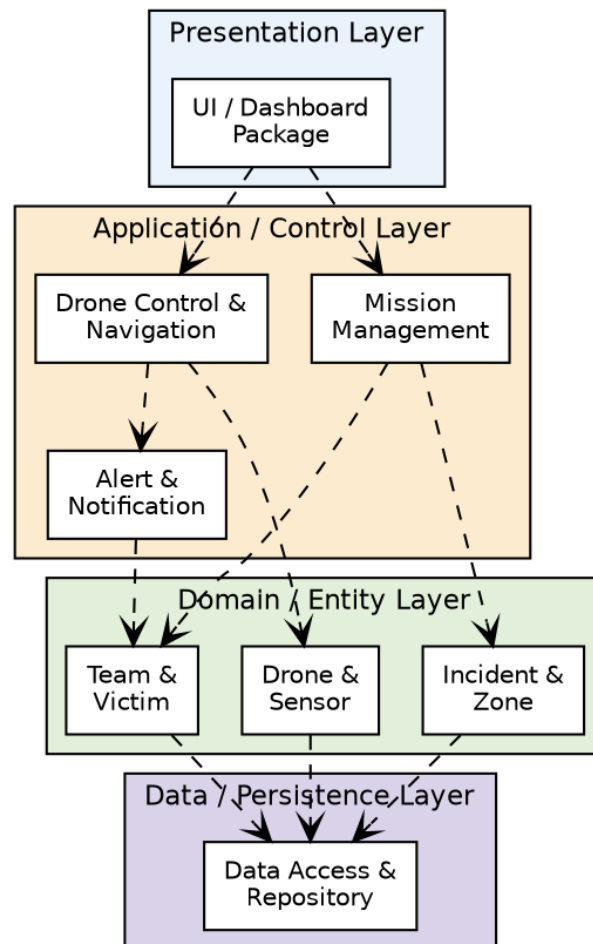


Figure 6: Package Diagram — ADDRRS

11. Component Diagram

The component diagram identifies the deployable/replaceable software components and the provided/required interfaces that connect them, reflecting the package structure above at an implementation level.

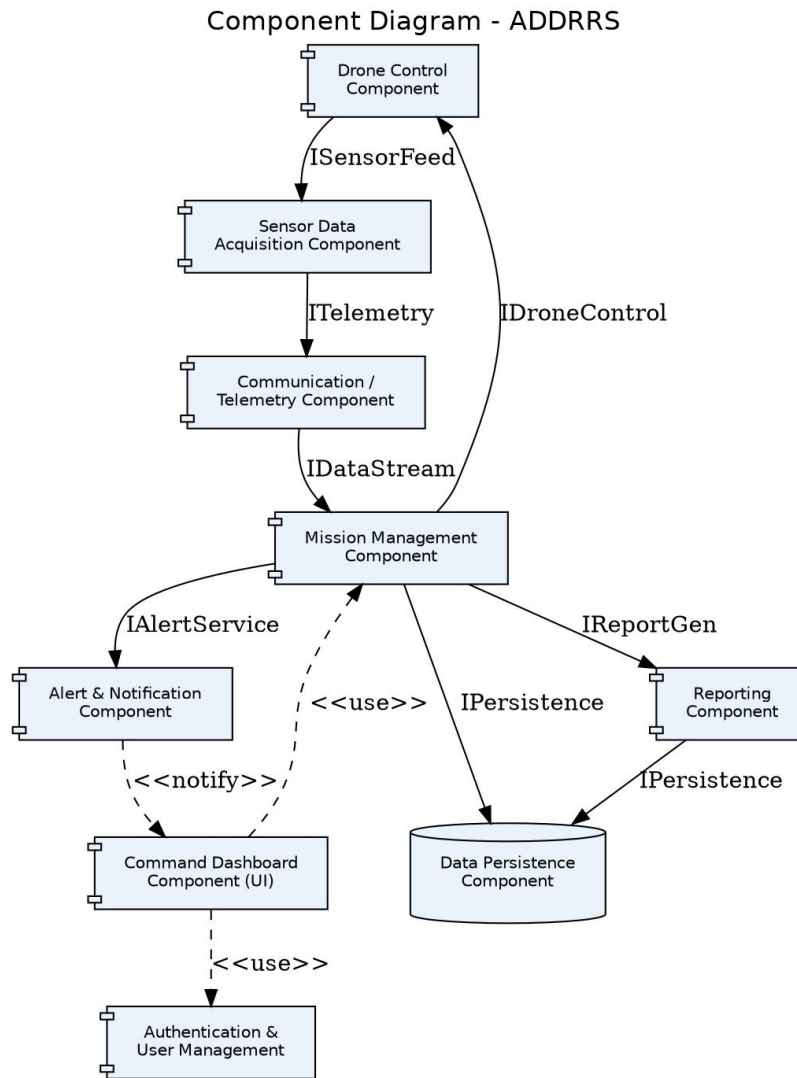


Figure 7: Component Diagram — ADDRRS

12. Deployment Diagram

The deployment diagram shows the physical distribution of the software across hardware nodes: the onboard drone flight controller, the cloud-hosted application and database servers, operator workstations, and field mobile devices, along with their communication protocols.

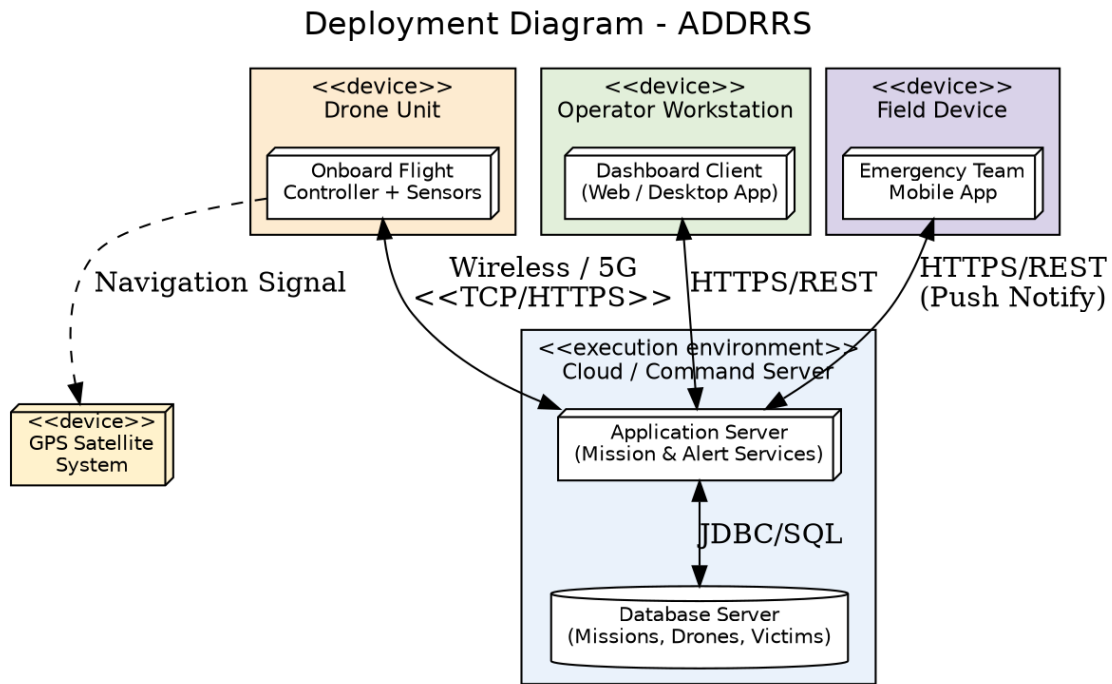


Figure 8: Deployment Diagram — ADDRRS

13. Object Responsibility Analysis (CRC Cards)

Class-Responsibility-Collaborator (CRC) analysis was used to assign responsibilities to objects while keeping coupling low and cohesion high.

Class	Responsibilities	Collaborators
Drone	Navigate to zone; capture and transmit sensor data; manage own flight/fault state; return to base autonomously.	Sensor, Route, CommandCenter, RescueMission
CommandCenter	Coordinate missions centrally; analyse incoming data; trigger alerts; act as single point of coordination (Singleton).	RescueMission, Alert, Drone
RescueMission	Track mission lifecycle; link zone, drone, and team; produce completion report.	Drone, EmergencyTeam, DisasterZone, MissionReport
EmergencyTeam	Group team members; accept dispatch instructions; report rescue outcome.	EmergencyTeamMember, RescueMission
Alert	Encapsulate alert data; notify relevant actors of a detected victim or hazard.	SensorData, EmergencyTeam

Class	Responsibilities	Collaborators
Sensor	Encapsulate sensing behaviour independent of drone logic (delegation, not drone doing everything itself).	Drone, SensorData

Responsibility was deliberately delegated away from the Drone class for sensing (to Sensor subclasses) and away from RescueMission for centralized coordination (to the CommandCenter singleton), avoiding a 'God Class' anti-pattern and keeping each class's reason to change singular.

14. Design Axioms, Principles and Design Pattern Justification

14.1 Design Axioms (Suh's Axiomatic Design)

- Independence Axiom: Functional requirements are kept independent — e.g., route planning (Route class) is decoupled from drone hardware control (Drone class), so route algorithms can change without affecting flight-control logic.
- Information Axiom: Among independent designs, the one with the least information content (simplest, most direct class structure) was preferred — e.g., a single abstract Sensor class with two concrete subclasses rather than a separate hierarchy per sensor attribute.

14.2 Design Principles Applied

- High Cohesion: Each class (e.g., Alert, Route, MissionReport) has a single, focused responsibility.
- Low Coupling: Layered package structure (Section 10) and interface-based component contracts (Section 11) minimise direct dependencies between layers.
- Encapsulation: Internal state (e.g., Drone.batteryLevel) is accessed only via defined methods.
- Abstraction: Sensor and User are abstract classes exposing a common contract to their subclasses.
- Liskov Substitution: Camera and EnvironmentalSensor can be used interchangeably wherever a Sensor is expected, since both fulfil the capture() contract.

14.3 Design Patterns Applied

Pattern	Application in ADDRRS	Justification
Singleton	CommandCenter	Ensures a single, globally consistent coordination point for all missions and alerts, avoiding conflicting mission state.
Observer	CommandCenter notifies Alert/EmergencyTeam/UI on new SensorData or Alert events	Decouples data producers (drones/sensors) from multiple interested consumers (UI, team dispatch, logging) — supports the extensibility requirement.
State	Drone lifecycle (Idle, Assigned, In-Flight, Fault, Returning, Charging)	Encapsulates state-specific behaviour and legal transitions, avoiding large conditional blocks and directly modelling the State Transition Diagram (Section 9).

Pattern	Application in ADDRRS	Justification
Strategy	Route.calculateOptimalPath()	Allows interchangeable route-planning algorithms (e.g., shortest-path vs. hazard-avoiding) to be selected at runtime without modifying the Drone or RescueMission classes.
Factory Method	Creation of Sensor subclass instances when a Drone is provisioned	Centralises and hides sensor object creation logic from client code, supporting new sensor types with minimal change (Open/Closed Principle).

15. Assumptions and Constraints

15.1 Assumptions

- Drones are pre-registered in the system with known sensor configurations before mission assignment.
- A reliable, if intermittent, wireless communication link exists between drones and the command server; brief outages are tolerated via onboard buffering.
- Emergency personnel carry a mobile device capable of receiving push notifications.
- GPS/navigation and weather data are available from trusted external systems.

15.2 Constraints

- The system boundary excludes physical drone hardware manufacturing and low-level flight-control firmware; only software-level coordination is modelled.
- All UML diagrams must remain consistent with the identified use cases (traceability enforced — Section 16).
- Real-time responsiveness is bounded by wireless network conditions in disaster-affected areas.
- Security constraints require role-based authentication for all human actor interactions with the system.

16. Requirement-to-Use Case/Class Traceability

The table below demonstrates traceability from key functional requirements through the responsible use case(s) to the primary implementing class(es), ensuring consistency across all UML views.

Requirement	Use Case	Primary Class(es)
Register disaster incidents	UC-01	DisasterIncident, EmergencyCoordinator
Define affected zones	UC-01	DisasterZone
Assign & dispatch drones	UC-02	RescueMission, Drone
Plan flight routes	UC-02	Route

Requirement	Use Case	Primary Class(es)
Capture & transmit sensor data	UC-03	Sensor, Camera, EnvironmentalSensor, SensorData, Drone
Detect victims & raise alerts	UC-03	CommandCenter, Alert, Victim
Coordinate & dispatch teams	UC-04	EmergencyTeam, EmergencyTeamMember
Generate mission reports	UC-04	MissionReport, RescueMission
Handle drone faults / return-to-base	UC-05	Drone (State pattern)
Authenticate & manage users	(Authenticate User)	User, EmergencyCoordinator, DroneOperator, EmergencyTeamMember

17. Implementation / Prototype and Results

To demonstrate the feasibility of the design, a lightweight object-oriented Python prototype (skeleton) was implemented to exercise the core classes and the Drone state-machine behaviour described in Section 9. The prototype is illustrative only — it demonstrates that the class responsibilities and state transitions defined in the model are directly implementable, and is included in the project's GitHub repository (Section 19) together with a short console-based simulation run.

```
class Drone:
    def __init__(self, drone_id):
        self.drone_id = drone_id
        self.status = "Idle"
        self.battery_level = 100.0

    def assign_mission(self, mission):
        if self.status == "Idle":
            self.status = "Assigned"
            mission.assign_drone(self)

    def take_off(self):
        if self.status in ("Assigned", "RoutePlanning"):
            self.status = "In-Flight"

    def transmit_data(self, sensor_data):
        if self.status == "In-Flight":
            self.status = "TransmittingData"
            return CommandCenter.get_instance().receive_data(self, sensor_data)

    def return_to_base(self):
        self.status = "Returning"

class CommandCenter:
    _instance = None

    @staticmethod
    def get_instance():
        if CommandCenter._instance is None:
            CommandCenter._instance = CommandCenter()
```

```

        return CommandCenter._instance

    def receive_data(self, drone, sensor_data):
        if sensor_data.get("victim_detected"):
            return Alert(sensor_data["location"]).notify()
        drone.status = "In-Flight"
        return "No victim detected -- continuing scan"

class Alert:
    def __init__(self, location):
        self.location = location

    def notify(self):
        return f"ALERT: possible victim at {self.location} -- team notified"

```

Result: a simulated run — creating a Drone, assigning it to a mission, taking off, and transmitting sensor data flagged with `victim_detected = True` — correctly transitioned the drone through `Idle` → `Assigned` → `In-Flight` → `TransmittingData` and produced the expected alert notification, confirming that the class responsibilities and state model in Sections 6 and 9 are internally consistent and implementable.

18. Reflection on Design Decisions and Learning Outcomes

Developing the OOAD model for ADDRRS demonstrated the importance of iterative refinement between structural and behavioural views: several classes identified during the initial noun-phrase analysis (Section 1.3) were later merged or reassigned responsibilities after the sequence and activity diagrams exposed missing collaborators, such as the need for a distinct `Route` class and a centralized `CommandCenter` coordinator. Applying Suh's design axioms in particular clarified why the `Sensor` hierarchy should be decoupled from `Drone` rather than embedding capture logic directly in the `Drone` class — a decision that also simplified applying the `Factory Method` pattern.

The exercise also highlighted the practical tension between designing for autonomous, real-time systems (which benefit from the `State` and `Observer` patterns for responsiveness and decoupling) and keeping the class model simple enough to remain traceable to the original requirements. Maintaining the traceability matrix (Section 16) throughout the exercise, rather than only at the end, was the most effective safeguard against diagrams drifting out of consistency with each other.

Relevance to Sustainable Development Goals

- SDG 3 (Good Health and Well-being): faster, more accurate victim location directly improves survival outcomes in disaster response.
- SDG 9 (Industry, Innovation and Infrastructure): the system demonstrates applied innovation in autonomous systems and resilient infrastructure.
- SDG 11 (Sustainable Cities and Communities): supports community resilience and coordinated emergency management.
- SDG 13 (Climate Action): supports rapid response to climate-related disasters such as floods and wildfires.